Lab Exercises

1. Consider the following snapshot of the system. Write C/C++ program to implement Banker's algorithm for deadlock avoidance. The program has to accept all inputs from the user. Assume the total number of instances of A,B and C are 10,5 and 7 respectively.

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

(a) What is the content of the matrix *Need*?
(b) Is the system in a safe state?
(c) If a request from process P1 arrives for (1, 0, 2), can the request be granted immediately? Display the updated Allocation, Need and Available matrices.
(d) If a request from process P4 arrives for (3, 3, 0), can the request be granted immediately?
(e) If a request from process P0 arrives for (0, 2, 0), can the request be granted immediately?

osl-4cce-d2@sce-cl12-32: ~/Documents/240953190/Week_8

```
osl-4cce-d2@sce-cl12-32:~/Documents/240953190/Week_8$ ./qn1
Enter number of processes: 5
Enter number of resource types: 3
Enter Total Instances of resources: 10 5 7
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Process Allocation      Need
P0      0 1 0           7 4 3
P1      2 0 0           1 2 2
P2      3 0 2           6 0 0
P3      2 1 1           0 1 1
P4      0 0 2           4 3 1

Available Resources:    3 3 2

System is in a SAFE STATE.
Safe Sequence: P1 P3 P4 P0 P2

--- Resource Request ---
Enter Process ID making request: 1
Enter Request Vector: 1 0 2

Request GRANTED immediately.

Process Allocation      Need
P0      0 1 0           7 4 3
P1      3 0 2           0 2 0
P2      3 0 2           6 0 0
P3      2 1 1           0 1 1
P4      0 0 2           4 3 1

Available Resources:    2 3 0
osl-4cce-d2@sce-cl12-32:~/Documents/240953190/Week_8$
```

osl-4cce-d2@sce-cl12-32: ~/Documents/240953190/Week_8

```
osl-4cce-d2@sce-cl12-32:~/Documents/240953190/Week_8$ ./qn1
Enter number of processes: 5
Enter number of resource types: 3
Enter Total Instances of resources: 10
5
7
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Process Allocation      Need
P0      0 1 0           7 4 3
P1      2 0 0           1 2 2
P2      3 0 3           6 0 -1
P3      2 1 1           0 1 1
P4      0 0 2           4 3 1

Available Resources:    3 3 1

System is in a SAFE STATE.
Safe Sequence: P3 P4 P1 P2 P0

--- Resource Request ---
Enter Process ID making request: 4
Enter Request Vector: 3 3 0

Request CANNOT be granted. It leads to an unsafe state. Rolling back...
osl-4cce-d2@sce-cl12-32:~/Documents/240953190/Week_8$
```

osl-4cce-d2@sce-cl12-32: ~/Documents/240953190/Week_8

```
osl-4cce-d2@sce-cl12-32:~/Documents/240953190/Week_8$ ./qn1
Enter number of processes: 5
Enter number of resource types: 3
Enter Total Instances of resources: 10 5 7
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Process Allocation      Need
P0      0 1 0           7 4 3
P1      2 0 0           1 2 2
P2      3 0 3           6 0 -1
P3      2 1 1           0 1 1
P4      0 0 2           4 3 1

Available Resources:    3 3 1

System is in a SAFE STATE.
Safe Sequence: P3 P4 P1 P2 P0

--- Resource Request ---
Enter Process ID making request: 0
Enter Request Vector: 0 2 0

Request GRANTED immediately.

Process Allocation      Need
P0      0 3 0           7 2 3
P1      2 0 0           1 2 2
P2      3 0 3           6 0 -1
P3      2 1 1           0 1 1
P4      0 0 2           4 3 1

Available Resources:    3 1 1
osl-4cce-d2@sce-cl12-32:~/Documents/240953190/Week_8$
```

```c
#include<stdio.h>
#define MAX 10
int n,m;

void needMatrix(int need[MAX][MAX],int max[MAX][MAX],int allot[MAX][MAX]){
for(int i=0;i<n;i++){
for(int j=0;j<m;j++){
need[i][j]=max[i][j]-allot[i][j];
}
}
}

int isSafe(int avail[],int max[MAX][MAX],int allot[MAX][MAX]){
int need[MAX][MAX];
needMatrix(need,max,allot);
int finish[MAX]={0};
int safeSeq[MAX];
int work[MAX];
int j;
for(int i=0;i<m;i++){
work[i]=avail[i];
}
int count=0;
while(count<n){
int found=0;
for(int i=0;i<n;i++){
if(!finish[i]){
for(j=0;j<m;j++){
if(need[i][j]>work[j]){
break;
}
}
if(j==m){
for(int k=0;k<m;k++){
work[k]+=allot[i][k];
}
safeSeq[count++]=i;
finish[i]=1;
found=1;
}
}
```

```c
if(!found){
printf("\nSystem is NOT in a safe state.\n");
return 0;
}
}
printf("\nSystem is in a safe state.\n");
printf("The safe sequence is: ");
for(int i=0;i<n;i++){
printf("P%d",safeSeq[i]);
if(i<n-1)printf("->");
}
printf("\n");
return 1;
}

void requestResources(int process,int request[],int avail[],int max[MAX][MAX],int allot[MAX][MAX]){
int need[MAX][MAX];
needMatrix(need,max,allot);
for(int i=0;i<m;i++){
if(request[i]>need[process][i]){
printf("\nRequest exceeds the need.\n");
return;
}
}
for(int i=0;i<m;i++){
if(request[i]>avail[i]){
printf("\nRequest exceeds the available number of resources.\n");
return;
}
}
for(int i=0;i<m;i++){
avail[i]-=request[i];
allot[process][i]+=request[i];
}
if(isSafe(avail,max,allot)){
printf("Request granted.\n");
}else{
printf("Request not granted.(unsafe state)\n");
for(int i=0;i<m;i++){
avail[i]+=request[i];
allot[process][i]-=request[i];
}
```

```
int main(){

int avail[MAX],max[MAX][MAX],allot[MAX][MAX],need[MAX][MAX];
int avail_orig[MAX],allot_orig[MAX][MAX];
int process,request[MAX];

printf("Enter the number of processes: ");
scanf("%d",&n);

printf("Enter the number of resource types: ");
scanf("%d",&m);

printf("\nEnter Allocation Matrix:\n");
for(int i=0;i<n;i++)
for(int j=0;j<m;j++)
scanf("%d",&allot[i][j]);

printf("\nEnter Max Matrix:\n");
for(int i=0;i<n;i++)
for(int j=0;j<m;j++)
scanf("%d",&max[i][j]);

printf("\nEnter Available Resources:\n");
for(int i=0;i<m;i++)
scanf("%d",&avail[i]);

for(int i=0;i<m;i++)
avail_orig[i]=avail[i];
for(int i=0;i<n;i++)
for(int j=0;j<m;j++)
allot_orig[i][j]=allot[i][j];

printf("\n(a) Need Matrix:\n");
needMatrix(need,max,allot);
for(int i=0;i<n;i++){
printf("P%d: ",i);
for(int j=0;j<m;j++)
printf("%d ",need[i][j]);
printf("\n");
}
```

Consider the following snapshot of the system. Write C/C++ program to implement deadlock detection algorithm.

(a) Is the system in a safe state?

(b) Suppose that process P2 make one additional request for instance of type C, can the system still be in a safe state?

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

```
tapash@Tapash: /mnt/c/Users

tapash@Tapash:/mnt/c/Users/upadh/OneDrive/Desktop/OS Lab/Week8$ ./qn2
Enter the number of processes: 5
Enter the number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Enter Request Matrix:
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2

Enter Available Resources:
0 0 0

(a)
No deadlock. System is in a safe state.

(b) P2 makes additional request for 1 instance of C:

Deadlock detected!
Processes in deadlock: P1 P2 P3 P4
tapash@Tapash:/mnt/c/Users/upadh/OneDrive/Desktop/OS Lab/Week8$
```

```
#include<stdio.h>
#define MAX 10
int n,m;

int isDeadlock(int avail[],int request[MAX][MAX],int allot[MAX][MAX]){

int work[MAX];
int finish[MAX];
int j;

for(int i=0;i<m;i++)
work[i]=avail[i];

for(int i=0;i<n;i++){
finish[i]=1;
for(int j=0;j<m;j++){
if(allot[i][j]!=0){
finish[i]=0;
break;
}
}
}

int count=0;
while(count<n){
int found=0;
for(int i=0;i<n;i++){
if(!finish[i]){
for(j=0;j<m;j++){
if(request[i][j]>work[j]){
break;
}
}
if(j==m){
for(int k=0;k<m;k++){
work[k]+=allot[i][k];
}
finish[i]=1;
found=1;
count++;
}
```

```
count++;
}
}
}
if(!found){
break;
}
}

int deadlock=0;
for(int i=0;i<n;i++){
if(!finish[i]){
deadlock=1;
break;
}
}

if(deadlock){
printf("\nDeadlock detected!\n");
printf("Processes in deadlock: ");
for(int i=0;i<n;i++){
if(!finish[i])
printf("P%d ",i);
}
printf("\n");
return 1;
}else{
printf("\nNo deadlock. System is in a safe state.\n");
return 0;
}

}

int main(){

int avail[MAX],request[MAX][MAX],allot[MAX][MAX];

printf("Enter the number of processes: ");
scanf("%d",&n);

printf("Enter the number of resource types: ");
scanf("%d",&m);
```

```
}

int main(){

int avail[MAX],request[MAX][MAX],allot[MAX][MAX];

printf("Enter the number of processes: ");
scanf("%d",&n);

printf("Enter the number of resource types: ");
scanf("%d",&m);

printf("\nEnter Allocation Matrix:\n");
for(int i=0;i<n;i++)
for(int j=0;j<m;j++)
scanf("%d",&allot[i][j]);

printf("\nEnter Request Matrix:\n");
for(int i=0;i<n;i++)
for(int j=0;j<m;j++)
scanf("%d",&request[i][j]);

printf("\nEnter Available Resources:\n");
for(int i=0;i<m;i++)
scanf("%d",&avail[i]);

printf("\n(a)");
isDeadlock(avail,request,allot);

printf("\n(b) P2 makes additional request for 1 instance of C:\n");
request[2][2]+=1;
isDeadlock(avail,request,allot);

return 0;

}
```

Lab – 9 Qn1

Write a C/C++ program to simulate First-fit, Best-fit and Worst-fit strategies. Given memory partitions of 100K, 500K, 200K, 300K, and 600K(in order), how would each of the First-fit, Best-fit, and Worst-fit algorithms place processes of 212K, 417K, 112K, and 426K (in order)? Which algorithm makes efficient use of memory?

```
tapash@Tapash: /mnt/c/Users

tapash@Tapash:/mnt/c/Users/upadh/OneDrive/Desktop/OS Lab/Week8$
tapash@Tapash:/mnt/c/Users/upadh/OneDrive/Desktop/OS Lab/Week8$
gcc lab9q1.c -o qn1
tapash@Tapash:/mnt/c/Users/upadh/OneDrive/Desktop/OS Lab/Week8$
./qn1
Memory Partitions: 100K 500K 200K 300K 600K
Processes: 212K 417K 112K 426K

First Fit:
Process         Size            Partition
P1              212K            P2(288K remaining)
P2              417K            P5(183K remaining)
P3              112K            P2(176K remaining)
P4              426K            Not Allocated

Best Fit:
Process         Size            Partition
P1              212K            P4(88K remaining)
P2              417K            P2(83K remaining)
P3              112K            P3(88K remaining)
P4              426K            P5(174K remaining)

Worst Fit:
Process         Size            Partition
P1              212K            P5(388K remaining)
P2              417K            P2(83K remaining)
P3              112K            P5(276K remaining)
P4              426K            Not Allocated

Best Fit is most efficient as it allocates all processes with le
ast wasted memory.
tapash@Tapash:/mnt/c/Users/upadh/OneDrive/Desktop/OS Lab/Week8$
```

```
#include<stdio.h>
#define MAX 10

void firstFit(int partitions[],int psize,int processes[],int prsize){

int alloc[MAX];
int remain[MAX];

for(int i=0;i<psize;i++){
remain[i]=partitions[i];
alloc[i]=-1;
}

printf("\nFirst Fit:\n");
printf("Process\t\tSize\t\tPartition\n");

for(int i=0;i<prsize;i++){
int allocated=0;
for(int j=0;j<psize;j++){
if(remain[j]>=processes[i]){
alloc[i]=j+1;
remain[j]-=processes[i];
allocated=1;
break;
}
}
if(allocated)
printf("P%d\t\t%dK\t\tP%d(%dK remaining)\n",i+1,processes[i],alloc[i],remain[alloc[i]-1]);
else
printf("P%d\t\t%dK\t\tNot Allocated\n",i+1,processes[i]);
}
}

void bestFit(int partitions[],int psize,int processes[],int prsize){

int alloc[MAX];
int remain[MAX];

for(int i=0;i<psize;i++){
remain[i]=partitions[i];
alloc[i]=-1;
```

```
printf("\nBest Fit:\n");
printf("Process\t\tSize\t\tPartition\n");

for(int i=0;i<prsize;i++){
int bestIdx=-1;
for(int j=0;j<psize;j++){
if(remain[j]>=processes[i]){
if(bestIdx==-1||remain[j]<remain[bestIdx]){
bestIdx=j;
}
}
}
if(bestIdx!=-1){
alloc[i]=bestIdx+1;
remain[bestIdx]-=processes[i];
printf("P%d\t\t%dK\t\tP%d(%dK remaining)\n",i+1,processes[i],alloc[i],remain[bestIdx]);
}else{
printf("P%d\t\t%dK\t\tNot Allocated\n",i+1,processes[i]);
}
}
}

void worstFit(int partitions[],int psize,int processes[],int prsize){

int alloc[MAX];
int remain[MAX];

for(int i=0;i<psize;i++){
remain[i]=partitions[i];
alloc[i]=-1;
}

printf("\nWorst Fit:\n");
printf("Process\t\tSize\t\tPartition\n");

for(int i=0;i<prsize;i++){
int worstIdx=-1;
for(int j=0;j<psize;j++){
if(remain[j]>=processes[i]){
if(worstIdx==-1||remain[j]>remain[worstIdx]){
worstIdx=j;
}
```

```
}
if(worstIdx!=-1){
alloc[i]=worstIdx+1;
remain[worstIdx]-=processes[i];
printf("P%d\t\t%dK\t\tP%d(%dK remaining)\n",i+1,processes[i],alloc[i],remain[worstIdx]);
}else{
printf("P%d\t\t%dK\t\tNot Allocated\n",i+1,processes[i]);
}
}
}

int main(){

int partitions[]={100,500,200,300,600};
int processes[]={212,417,112,426};
int psize=5;
int prsize=4;

printf("Memory Partitions: 100K 500K 200K 300K 600K");
printf("\nProcesses: 212K 417K 112K 426K\n");

firstFit(partitions,psize,processes,prsize);

bestFit(partitions,psize,processes,prsize);

worstFit(partitions,psize,processes,prsize);

printf("\nBest Fit is most efficient as it allocates all processes with least wasted memory.\n");

return 0;

}
```